Apunte Docente: Introducción a Backtracking

Asignatura: Programación III **Unidad:** Técnicas de Diseño de Algoritmos **Nivel:** Universitario

1. ¿QUÉ ES BACKTRACKING?

El término **Backtracking** (en español, *Vuelta Atrás*) define un esquema algorítmico sistemático para resolver problemas mediante la **búsqueda exhaustiva e inteligente en un espacio de soluciones**.

Consiste en construir una solución paso a paso. Si en un punto determinado se detecta que la secuencia actual de decisiones no puede completarse para formar una solución válida, el algoritmo **retrocede** (hace *backtrack*) al paso anterior y prueba una alternativa diferente.

> **IDEA CLAVE**
> Es una búsqueda en profundidad (*Depth-First Search - DFS*) organizada sobre un árbol implícito de posibilidades, reforzada con un mecanismo de **poda** (*pruning*) para evitar explorar caminos no viables.

2. ANALOGÍA: EL LABERINTO

Imaginen que están recorriendo un laberinto a ciegas:

1. Avanzan por un pasillo hasta llegar a un cruce con varias opciones.
2. Eligen un camino arbitrario y avanzan por él.
3. Si llegan a un **callejón sin salida**, retroceden hasta la última intersección y eligen una opción no explorada.
4. Repiten el proceso hasta encontrar la salida o agotar las opciones.

3. ELEMENTOS FUNDAMENTALES

Para modelar un problema mediante Backtracking, debemos definir tres elementos:

1. Tupla Solución
Representada como $(x_1, x_2, ..., x_n)$. Define la secuencia de decisiones tomada en cada etapa i.

2. Espacio de Búsqueda
Árbol de decisión implícito donde la raíz es el estado inicial y los nodos representan estados parciales.

3. Función de Poda
Reglas que determinan si un estado parcial puede derivar en una solución válida. Si falla, se corta la rama.

4. ESQUEMA GENERAL DE IMPLEMENTACIÓN

Estructura general recursiva para encontrar soluciones en un espacio de búsqueda:

```
Procedimiento Backtracking(etapa, solución_parcial):
    Si es_solución_completa(etapa, solución_parcial) Entonces
        procesar_o_guardar(solución_parcial)
        Retornar
    FinSi

    Para cada opción en obtener_opciones_válidas(etapa) Hacer
        solución_parcial[etapa] <- opción

        // PODA: Solo avanzamos si el estado actual es prometedor
        Si es_prometedor(etapa, solución_parcial) Entonces
            Backtracking(etapa + 1, solución_parcial)
        FinSi

        deshacer_elección(etapa, solución_parcial) // Vuelta Atrás
    FinPara
FinProcedimiento
```

5. EJEMPLO CLÁSICO: EL PROBLEMA DE LAS N-REINAS

Enunciado: Colocar N reinas de ajedrez en un tablero de $N \times N$ de forma que ninguna reina amenace a otra (no deben compartir fila, columna ni diagonal).

- **Decisión x_i:** Fila en la que se ubica la reina en la columna i.
- **Etapa i:** Columna actual ($0 \leq i < N$).
- **Condición de Poda:** La posición $(fila, i)$ no debe sufrir ataques de las reinas ya colocadas en las columnas previas.

Código de Referencia en Java

```
public class NReinas {
    private int n;
    private int[] reinas; // reinas[columna] = fila

    public NReinas(int n) {
        this.n = n;
        this.reinas = new int[n];
    }

    public void resolver() {
        buscarSoluciones(0);
    }

    private void buscarSoluciones(int columna) {
        if (columna == n) {
            mostrarTablero();
            return;
        }

        for (int fila = 0; fila < n; fila++) {
            if (esSeguro(fila, columna)) {
                reinas[columna] = fila;        // Tomar decisión
                buscarSoluciones(columna + 1); // Avanzar a la siguiente etapa
                // Deshacer decisión: en este array la posición se
                // sobrescribe en la siguiente iteración del bucle.
            }
        }
    }

    private boolean esSeguro(int fila, int columna) {
        for (int c = 0; c < columna; c++) {
            int f = reinas[c];
            if (f == fila || Math.abs(f - fila) == Math.abs(c - columna)) {
                return false; // PODA: Misma fila o diagonal
            }
        }
        return true;
    }
}
```

6. COMPLEJIDAD COMPUTACIONAL Y RENDIMIENTO

- **Peor Caso:** Exponencial (ej. $O(N!)$ o $O(2^N)$), equivalente a la exploración por fuerza bruta.
- **Caso Promedio:** La complejidad real depende fuertemente de la **eficiencia de la función de poda**. Una buena función elimina subárboles enteros con millones de nodos, haciendo computable el problema en fracciones de segundo.

7. PREGUNTAS DE AUTOEVALUACIÓN

1. ¿Cuál es la diferencia principal entre un enfoque de Fuerza Bruta convencional y un esquema de Backtracking?